



DIPARTIMENTO DI **I**NGEGNERIA **E**LETTRICA E **T**ECNOLOGIE DELL'**I**NFORMAZIONE
SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

LAUREA IN INGEGNERIA DELL'AUTOMAZIONE

VLM-Driven Neuro-Symbolic Task and Motion Planning with Ambiguity Resolution

Candidato:
Vincenzo Palomba

Relatori:
Prof. Alberto Finzi

Academic Year 2025/2026

Outline

1. Introduction
2. Deliberative Layer
3. Executive Layer
4. Reactive Layer
5. Setup and Experimental Results
6. Conclusion

Project Objective

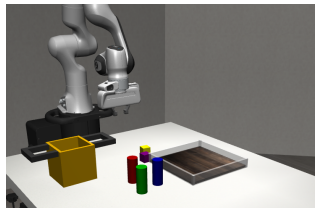
A Neuro-Symbolic Task and Motion Planning (TAMP) architecture bridging natural language reasoning with reactive physical execution.

Core Features

- ▶ **Ambiguity Resolution:** Proactively identify and resolve referential ambiguities in human prompts (e.g., "*Which cube?*") via multi-turn visual dialogue prior to physical execution.
- ▶ **Symbolic Verification:** Generate PDDL plans from the semantic perception pipeline in order to guarantee causal correctness and eliminate VLM hallucinations.
- ▶ **Dynamic Behavior Tree:** Execute discrete symbolic plans using dynamically generated Behavior Trees, continuous condition monitoring, and auto-recovery fallbacks.
- ▶ **Hybrid Manipulation:** Combine Task-Space RRT for collision-free transit with RL (Proximal Policy Optimization) for compliant, contact-rich grasping.

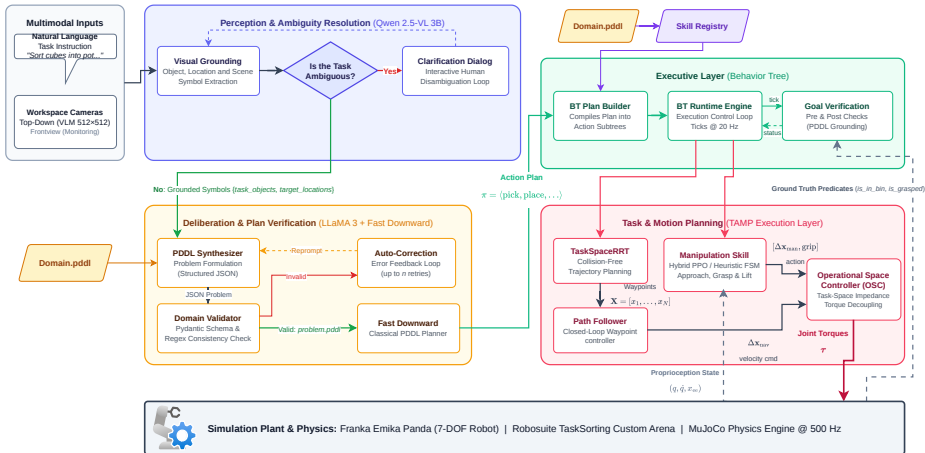
Scenario: *TaskSorting Arena*

The simulation environment is built on top of the Robosuite library with further customizations.



- ▶ **Robot Hardware:** 7-DoF Franka Emika Panda.
- ▶ **Scene Objects:** Cubes, cans and bins.
- ▶ **Physics Engine:** MuJoCo headless continuous simulation.
- ▶ **Perception:** Dual-view cameras (Top-Down + Frontal).

Neuro-Symbolic Planning architecture



Perception: Visual Grounding & Ambiguity Resolution

Grounds natural language human instructions in the observed scene and explicitly reasons about task ambiguity through multi-turn visual dialogue. It eliminates hallucinated assumptions before symbolic deliberation

Ambiguity Resolution Pipeline

► VLM Visual Scene Grounding (Qwen 2.5-VL):

Parses the camera frame into entity schemas:

- task_objects: Manipulable items (e.g., cans, cubes).
- target_locations: Receptacles (e.g., sorting bin, pot).

► Ambiguity Detection: Evaluates query against detected scene entities:

- Flags referential ambiguity (e.g., multiple cubes present).
- Schema: AmbiguityReasoning(task_ambiguous, question).

► Human-in-the-Loop Clarification: Prompts the user with a targeted question, appends response to chat history, and disambiguates the scene.

► Symbols Grounding: Outputs clean symbols $\{\text{Objects, Locations}\} \subset O_{\text{obj}}, O_{\text{loc}}$ directly to the LLaMA-3 PDDL synthesizer.

Disambiguation Example

Human: "Put the cube in the container."

```
[Visual Scene Grounding]
task_objects: [yellow_cube, purple_cube]
target_locations: [sorting_bin, pot]
Ambiguity_Flagged: True
```

Robot: "Which cube should I move, the yellow or the purple one? And into the pot or the bin?"

Human: "The yellow cube into the pot."

```
[Symbols Grounding]
target: yellow_cube → (type obj)
destination: pot → (type location)
```

VLM Architectures: Dual-Encoder Foundations

Vision-Language Models (VLMs) integrate visual perception with linguistic reasoning by bridging continuous visual feature spaces and discrete language token spaces.

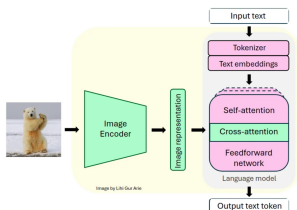


Figure: Standard Dual-Encoder VLM Architecture

Language Encoder (Transformer)

- ▶ Transforms discrete sequences into continuous numerical **embeddings**.
- ▶ Utilizes **self-attention mechanisms** to dynamically weigh token importance.
- ▶ **Decoders** auto-regressively generate the most statistically probable sequence.

Vision Encoder (CNN / ViT)

- ▶ Extracts hierarchical spatial features via Convolutional Neural Networks or Vision Transformers.
- ▶ ViTs segment images into non-overlapping patches, processing them as sequential tokens.

VLM Training Methodologies

Training fuses information from vision and language encoders, correlating visual features with text semantics to enable unified multimodal reasoning.

Contrastive Learning

Aligns modalities in a shared embedding space. The model minimizes the latent distance between matching image-text pairs while maximizing it for non-matching pairs.

Masked Modeling

Forces the model to reconstruct randomly obscured image patches or masked text tokens, learning robust contextual representations.

Generative Training

Optimizes the model to auto-regressively generate text conditioned on visual inputs (e.g., image captioning or visual question answering).

Pretrained Projection

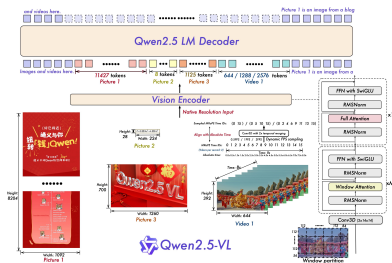
Freezes a pretrained LLM and Vision Encoder, training only a lightweight mapping network (e.g., MLP projector) to translate visual embeddings into the LLM's input space.

Qwen 2.5-VL Architecture

A state-of-the-art multimodal foundation model utilizing a dynamic-resolution Vision Transformer coupled with a robust language decoder.

Core Components

- ▶ **Qwen2.5 LM Decoder:** Advanced autoregressive language model for reasoning and generation.
- ▶ **Vision Transformer (ViT):** Uses 3D convolutions to efficiently process visual data into sequences of 14×14 patches.
- ▶ **Dynamic Preprocessor:** Adjusts image size dynamically to preserve native aspect ratios and extracts frames optimally for video inputs.



Design Rationale for this Architecture

- ▶ **Dynamic Resolution:** Preserves small object aspect ratios without distortion.
- ▶ **Local Real-Time Inference:** Runs locally via Ollama (1.25 s latency, zero API dependency).
- ▶ **High Spatial Fidelity:** Superior grounding of manipulable items and receptacle boundaries.

Symbolic Planning: PDDL Formulation

Symbolic Planning uses formal logic to find a sequence of actions that transition a system from an initial state to a target goal state. **PDDL** (Planning Domain Definition Language) is the standard modeling language for this, decoupling the robot's physical capabilities from specific task specifications to guarantee causal correctness before execution.

Formal Planning Tuple

$$\mathcal{P} = (\mathcal{D}, O, s_0, G)$$

► Planning Domain $\mathcal{D} = (\mathcal{L}, \mathcal{A})$ [Static]:

- $\mathcal{L}$: Predicates describing state relations.
- $\mathcal{A}$: Action schemas with preconditions and effects.
- *Fixed and verified offline* (robot capability bounds).

► Planning Problem Instance [Dynamic]:

- O : Grounded universe of objects and locations.
- s_0 : Completely grounded initial state from VLM.
- G : Target goal conditions from user task.
- *Synthesized online* via LLaMA 3 from perception.

Key Principle: Domain physics are fixed; only state and goal are queried online.

Manipulation Domain Definition (domain.pddl)

```
; Grounded State Predicates  $\mathcal{L}$ 
(holding ?obj)
(on-table ?obj)
(on ?obj ?loc)

; Robotic Action Schemas  $\mathcal{A}$ 
(:action pick
  :parameters (?o - obj)
  :precondition (and (on-table ?o)
                     (not (holding ?o)))
  :effect (and (holding ?o)
               (not (on-table ?o))))

(:action place
  :parameters (?o - obj ?loc - location)
  :precondition (holding ?o)
  :effect (and (on ?o ?loc)
               (not (holding ?o))))
```

Neuro-symbolic planning: PDDL Problem Synthesis

While domain physics $\mathcal{D}$ are fixed offline, the planning problem $\mathcal{P} = (\mathcal{D}, O, s_0, G)$ is formulated online. **LLaMA 3** translates visually grounded symbols and task intent into (O, s_0, G) to complete $\mathcal{P}$.

Problem Synthesis Workflow

- ▶ **1. Grounded Few-Shot Prompting:** Uses grounded entities ($O_{\text{obj}}, O_{\text{loc}}$), scene state s_0 , and task G alongside domain $\mathcal{D}$. Enforces strict typing ($O_{\text{obj}} \rightarrow \text{obj}$, $O_{\text{loc}} \rightarrow \text{location}$) and maps task goals to domain predicates.
- ▶ **2. Pydantic Validation, Auto-Repair & Compilation:** Validates LLM output against pydantic Problem schema, auto-correct undeclared symbols, and retries on predicate violations. `Problem.to_pddl()` deterministically serializes the verified instance into standard PDDL syntax.

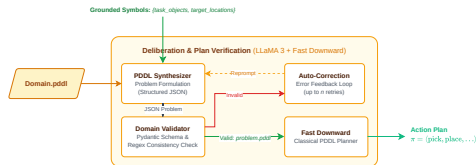
Key Guarantee: Structured JSON + Pydantic validation eliminates LLM syntax hallucinations before reaching the planner.

Generated Planning Problem (problem.pddl)

```
; Online problem instance  $\mathcal{P} = (\mathcal{D}, O, s_0, G)$ 
(define (problem manipulation-task)
  (:domain manipulation)
  (:objects
    red_can yellow_cube blue_can - obj
    sorting_bin pot - location
  )
  (:init
    (on-table red_can)
    (on-table yellow_cube)
    (on-table blue_can)
  )
  (:goal (and
    (on red_can sorting_bin)
    (on yellow_cube pot)
    (on blue_can sorting_bin)
  ))
)
```

Plan Verification: Fast Downward Solver

Fast Downward acts as the symbolic deliberation engine, using initial states s_0 , final goal G and Domain specification $\mathcal{D}$ to obtain a verified action sequences via **Heuristic Forward State-Space Search**.



Deliberation Guarantee: Computes verified causal action plans in < 0.08 s with zero hallucination.

Forward Search & Heuristics

- **Search Paradigm:** Progresses forward in state space $\mathcal{S}$:

$$s_0 \xrightarrow{a_1} s_1 \rightarrow \dots \rightarrow s_g \models G.$$

- **Direct Grounding:** Grounded s_0 from VLM makes forward state progression directly evaluable.
- **State Transitions:** Applies applicable action a ($\text{pre}(a) \subseteq s$):
 $s' = (s \setminus \text{del}(a)) \cup \text{add}(a).$
- **Heuristic Engine (LAMA / h_{FF}):**

- **Lazy GBFS:** Greedy best-first search for high-speed plan discovery.
- **Dual Heuristics ($h_{FF} + h_{LM}$):** Delete-relaxation ($\text{del}(a) = \emptyset$) + causal Landmarks.
- **Preferred Operators:** Prunes search branching via helpful actions.

PDDL Problem Generation experiments

To bridge perceptual symbols and symbolic planning, the deliberation layer couples **Pydantic data models** with programmatic domain validation and a reflection loop to eliminate hallucinations.

Structured Pipeline & Pydantic Schemas

- ▶ **Synthesizer Class** (`AmbresStructured`):
`generate_problem_json(...)` queries LLaMA 3 with $(s_0, G, O_{\text{obj}}, O_{\text{loc}}, \mathcal{D})$ under few-shot JSON constraints.
- ▶ **Data Models** (`src.llm2pddl.problem`):
 - ▶ **Problem**: Declares objects, init, and goal.
 - ▶ **PDDLObject**: Type enforcement (`obj, location`).
 - ▶ **Predicate**: Structured facts with arity validation.
- ▶ **Compilation** (`Problem.to_pddl()`):
Deterministically convert the JSON payload schema into standard PDDL formatted text.

Closed-Loop Validation & Auto-Repair

- ▶ **Validation Function**:
`generate_problem_with_retry(...,  $\mathcal{D}$ )` verifies predicates and types against $\mathcal{D}$ via regex extraction.
- ▶ **Handled Failure Modes**:
 - ▶ *Predicate Hallucination*: e.g., $(\text{in } ?x \text{ ?bin}) \notin \mathcal{D}$.
 - ▶ *Undeclared Symbols*: Missing object in objects.
 - ▶ *Type Inversion*: Receptacle labeled as obj.
- ▶ **Auto-Healing & Reflection**:
 - ▶ Inferred auto-declaration for missing receptacles.
 - ▶ Re-prompts LLaMA 3 with exact error traces (max 5 retries).

PDDL Problem Generation experiments

Empirical validation across **28 simulation trials** demonstrates that closed-loop PDDL synthesis achieves 100% feasibility and reliably shields the executive layer.

Empirical Benchmark

Evaluation Metric	Result
Total Logged Trials	28
1st-Pass Solver Feasibility	26 / 28 (92.9%)
Success after Feedback Loop	28 / 28 (100.0%)
Fast Downward Parser Errors	0% (0 / 28)
Avg. Symbolic Solve Time	0.084 s
Max Sequential Objects	4 (8 plan steps)

- ▶ **92.9% First-Pass Feasibility:** Raw LLaMA 3 output.
- ▶ **100% Closed-Loop Success:** Feedback loop repairs all edge cases within 1–2 reflection retries.

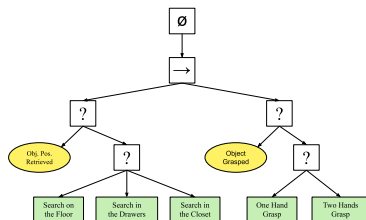
Impact on the Executive Layer

- ▶ **Causal Feasibility Verification:**
Fast Downward verifies task feasibility *prior* to actuation; unreachable goals fail safely offline.
- ▶ **Plan-to-BT Compiler:**
`build_bt_from_pddl_plan(plan, adapter)` compiles verified plans $\pi = \langle a_1, \dots, a_N \rangle$ into reactive Behavior Trees.
- ▶ **Executive Shielding:**
The Behavior Tree is completely shielded from malformed symbols, broken preconditions, or hallucinated actions.

Bridge to Execution: Verified symbolic deliberation guarantees an error-free plan handoff to reactive Behavior Tree execution.

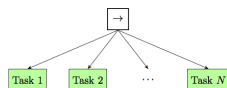
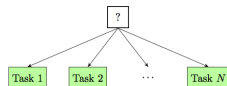
Executive Layer: Behavior Trees

- ▶ A **Behavior Tree** is a directed tree with nodes classified as root, control flow, or execution (tasks).
- ▶ Execution starts from the root, which sends *ticks* to its children at a set frequency.
- ▶ A node returns one of three statuses: **Running**, **Success**, or **Failure**.



Control Flow Nodes

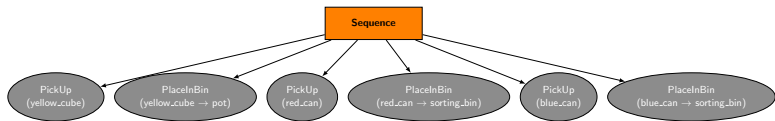
- ▶ **Selector:** A fallback node that executes the first child that does not fail. Returns **Success** or **Running**.
- ▶ **Sequence:** Executes children in order until one succeeds. Returns **Failure** or **Running** immediately if a child fails or is still running.



How to build Behavior Trees: `py_trees` library

`py_trees` is a Python library used to construct Behavior Trees efficiently. It relies primarily on two base classes: **Behaviours** and **Composites**.

- ▶ **Behaviour (Leaf):** The smallest element in a tree. Behaviours typically represent a **condition** (e.g., *Am I hungry?*) or an **action** (e.g., *Buy chocolate cookies*).
- ▶ **Composites (Control Flow):** Responsible for directing the path traced through the tree during a tick.
 - ▶ **Selectors:** Fallback decision-makers.
 - ▶ **Sequences:** Order-dependent factories.
 - ▶ **Parallels:** Concurrent execution factories.



BT Builder: Parsing and Sequence Compilation

1. Parsing PDDL Plan

(`parse_pddl_action`) Cleans raw planner text into structured Python data:

- ▶ **Filters Clutter:** Discards comments starting with ; (e.g., ; cost = 2) and blank lines.
- ▶ **Strips Planner Syntax:** Removes step prefixes (e.g., 0:) and outer parentheses (...).
- ▶ **Extracts Tokens:** Normalizes action name to lowercase and collects arguments:

```
"0: (place yellow_cube pot)" → ("place",  
    ["yellow_cube", "pot"])
```

2. Plan to Sequence

(`pddl_plan_to_sequence`) Converts the parsed action list into an executable Behavior Tree Sequence:

- ▶ **Skill Dispatch:** Uses `SKILL_REGISTRY` to map actions to robot skills ("pick", "place").
- ▶ **Argument Binding:** Passes simulator environment and object targets (object, asset).

Compilation Pipeline Example

1. Planner Output (Fast Downward):

```
; plan generated by fast downward  
0: (pick yellow_cube)  
1: (place yellow_cube pot)  
; cost = 2
```

⇓ *parsed & compiled into BT*

2. Executable PyTrees Sequence:

```
[~] pddl_plan_sequence (memory=True)  
|-- PickUp(object='yellow_cube')  
'-- PlaceInBin(object='yellow_cube',  
    asset='pot')
```

Design Rationale

Cleanly decouples **symbolic task planning** (Fast Downward) from the **physical robot control** (Motion planning).

BT Builder: Goal Checking and Tree Generation

3. Goal-Guarded Wrapper

(wrap_with_goal_check) A plain sequence is open-loop. If an object drops, it fails blindly. We wrap it in a reactive fallback loop:

- ▶ **Pre-Check (GoalCheck_Pre)**: Verifies if goal is already met before moving. If yes → succeeds immediately (no redundant actions).
- ▶ **Selector (memory=False)**: Re-evaluates continuously on every tick for dynamic preemption.
- ▶ **Post-Check (GoalCheck_Post)**: Confirms physical completion after the sequence finishes.
- ▶ **Retry Loop (Retry)**: If the grasp slips and post-check fails, automatically retries the plan up to N times.

4. Main Entrypoint

(build_bt_from_pddl_plan) Single builder API providing two execution modes:

- ▶ **wrap_goal_check=False**: Raw Sequence (open-loop baseline).
- ▶ **wrap_goal_check=True**: Self-healing tree with retry loop.

Goal-Guarded Tree Architecture

```
[?] goal_guarded_root (Selector)
|-- [>] GoalCheck_Pre (already met?)
'-- [R] retry_until_goal (max=10)
    '-- [->] attempt (Sequence)
        |-- [->] pddl_plan_sequence
        |   |-- PickUp(yellow_cube)
        |   '-- PlaceInBin(yellow_cube->pot)
        '-- [>] GoalCheck_Post (verified?)
```

Failure Recovery Example (Cube Drop)

1. Robot picks up cube, but gripper slips mid-air.
2. Sequence finishes → GoalCheck.Post checks pot.
3. Cube is missing → returns FAILURE.
4. Retry catches failure → re-runs the plan → task completed successfully!

BT Nodes: Skills & Conditions

The executive layer instantiates action skills and condition monitors as modular `py_trees` nodes, bridging discrete PDDL plans to continuous 20 Hz robot control.

Action Skills (`skills.py`)

- ▶ **Base Architecture (Skill):**
Subclasses `Behaviour`. Non-blocking
20 Hz tick: returns `RUNNING`, `SUCCESS` on `is_done()`, or `FAILURE` on timeout. Emits 7-DOF OSC commands $a \in \mathbb{R}^7$.
- ▶ **Pick-Up Primitive**
(`MotionPlanningPickUp`):
4-stage FSM: `follow_path` (RRT to hover) $\rightarrow$ `descend` (linear approach) $\rightarrow$ `grasp` (finger closure) $\rightarrow$ `lift` (clearance).
▶ *Hybrid PPO Variant*: Replaces descent/grasp with learned RL policy.
- ▶ **Place Primitive**
(`MotionPlanningPlaceInBin`):
2-stage FSM: `follow_path` (RRT transit with locked grip) $\rightarrow$ `release` (open gripper over container).

Condition Nodes (`conditions.py`)

- ▶ **Physical Grounding (conditions.py):**
Evaluates simulator state at zero latency:
 - `on(?x, ?y)  $\leftrightarrow$  env.is_in_bin(obj, target)`
 - `holding(?x)  $\leftrightarrow$  env.is_grasped(obj)`
 - `on-table(?x)  $\leftrightarrow$   $\neg$ grasped  $\wedge$   $\neg$ in_bin`
- ▶ **PDDL Goal Monitor (PDDLGoalCheck):**
Parses conjunctive goals $\bigwedge g_i$ from `problem.pddl`. Evaluates pre/post conditions to skip met goals or trigger retry.

Execution & Runtime Tree

The runtime engine executes the compiled Behavior Tree via a 20 Hz tick loop, continuously monitoring preconditions, dispatching skills, and recovering from physical disturbances.

Runtime Execution Pipeline

- ▶ **20 Hz Tick Loop**
(BehaviourTree.tick()):
Synchronously evaluates the tree to active leaf; steps simulation with 7-DOF OSC action $a \in \mathbb{R}^7$.
- ▶ **4-Phase Reactive Flow:**
 - 1. *Pre-Check*: GoalCheck_Pre skips met goals (0 redundant steps).
 - 2. *Dispatch*: Sequence executes skills in deterministic plan order.
 - 3. *Motion*: Active skill steps RRT waypoints at 20 Hz.
 - 4. *Recovery*: GoalCheck_Post triggers Retry on slip or drop.
- ▶ **Live State Introspection:**
Live HUD tracks active node, motion stage, and tick counter in real time.

Generated Behavior Tree Structure

```
[?] goal_guarded_root (Selector)
|-- [>] GoalCheck_Pre (already met?)
'-- [R] retry_until_goal (max=3)
    '-- [->] attempt (Sequence)
        |-- [->] pddl_plan_sequence
        |   |-- PickUp(yellow_cube)
        |   '-- PlaceInBin(cube->pot)
        '-- [>] GoalCheck_Post (verified?)
```

- ▶ **Selector [?]**: Dynamic fallback preemption.
- ▶ **Sequence [->]**: Preserves PDDL action order.
- ▶ **Retry [R]**: Autonomous loop on slip or drop.

Reactive Robustness: Closed-loop Behavior Trees autonomously intercept execution disturbances and trigger plan recovery.

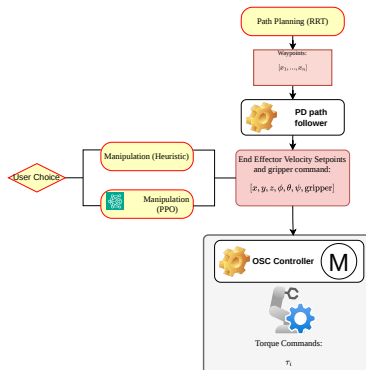
Reactive Layer: Motion and Path Planning

Reactive Layer Roles

- ▶ **Symbolic-to-Continuous Grounding:** Translates discrete BT skills (pick, place) into continuous robot motion.
- ▶ **Perception:** Accounts for physical geometry, obstacles, and contact dynamics ignored by PDDL.
- ▶ **Motor Control:** Operates at 20 Hz with real-time state feedback from MuJoCo Simulator Environment.

Hierarchical Motion Decomposition

- ▶ **1. Path Planning (Task Space RRT):** Solves pure geometric obstacle avoidance in free space ($\mathbb{R}^3$).
- ▶ **2. Trajectory Tracking:** Closed-loop proportional tracking of Cartesian waypoints $[x_1, \dots, x_n]$ generated by the RRT.
- ▶ **3. Manipulation (PPO / Heuristic):** Governs fine contact dynamics, compliant grasping, and lifting.
- ▶ **4. Low Level Controller (OSC):** Takes as input the cartesian velocities from the Tracking and Manipulation modules and output the joint torques τ_i needed to control the robot.



Path Planning: RRT

Rapidly Random Trees (RRT) is a Probabilistic Path Planning method used to efficiently construct a path in continuous spaces.

Algorithm Goal Oriented RRT Algorithm

```
1:  $G.\text{init}(x_{\text{start}})$ 
2: for  $i = 1 \dots K$  do
3:    $p = \text{UniformRandom}(0, 1)$ 
4:   if  $p \leq p_{\text{goal}}$  then
5:      $x_{\text{rand}} \leftarrow x_{\text{goal}}$ 
6:   else
7:      $x_{\text{rand}} \leftarrow \text{RandPose}()$ 
8:   end if
9:    $x_{\text{nearest}} \leftarrow \text{NearestVertex}(x_{\text{rand}}, G)$ 
10:   $x_{\text{new}} \leftarrow \text{NewConf}(x_{\text{nearest}}, x_{\text{rand}}, \Delta x)$ 
11:  if  $\text{CollisionFree}(x_{\text{nearest}}, x_{\text{new}})$  then
12:     $G.\text{add\_vertex}(x_{\text{new}})$ 
13:     $G.\text{add\_edge}(x_{\text{nearest}}, x_{\text{new}})$ 
14:    if  $x_{\text{new}} \in X_{\text{goal}}$  then return Path Found
15:  end if
16: end if
17: end for
18: return  $G$ 
```

The goal oriented version samples the goal point with a given probability p while a random point with probability $1 - p$.

Trajectory Tracking

- ▶ **Cartesian Setpoints:** The waypoints generated by the task-space RRT are Cartesian poses used as setpoints for a **Proportional (P) Controller**.
- ▶ **Tracking Error:** At each time step, the position error is defined as:

$$\mathbf{e} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{EF}}$$

- ▶ **Control Law:** The controller outputs velocity commands bounded by a maximum speed:

$$\mathbf{v}_{\text{cmd}} = \text{clip}(K_p \mathbf{e}, -v_{\text{max}}, v_{\text{max}})$$

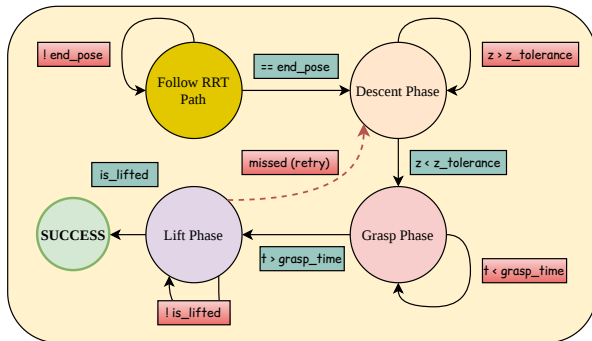
which are sent to the underlying operational space controller.

- ▶ **Application:** Used specifically during the **approach phase** in the pick action and during the **place action**. The place skill itself is completely based on the RRT planner and the trajectory tracker, while the pick skill is composed by more modules and need further explanation.

Manipulation: Heuristic Approach

Consists of a **Staged FSM**, that executes the following phases:

1. **RRT Planning:** Traverse collision-free waypoints to hover pose ($== \text{end_pose}$).
2. **Descent Phase:** Descend vertically until reaching grasp height ($z \leq z_{\text{tol}}$).
3. **Grasp Phase:** Actuate gripper firmly for set duration ($t > t_{\text{grasp}}$).
4. **Lift Phase:** Elevate object to safe height to verify pick. If it fails, it will retry from the Descent phase.



Proximal Policy Optimization

Proximal Policy Optimization is an on-policy actor-critic Reinforcement Learning method. PPO updates policy parameters θ_{k+1} by maximizing the **Clipped Surrogate Objective**:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where the probability ratio is defined as:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}$$

- ▶ **Clipped Objective:** The hyperparameter $\epsilon \approx 0.2$ penalizes moving $r_t(\theta)$ outside $[1 - \epsilon, 1 + \epsilon]$, preventing destructively large policy updates.
- ▶ **Actor-Critic Architecture:** Jointly optimizes a policy network $\pi_{\theta}(a|s)$ and a state-value baseline $V_{\phi}(s)$ using Generalized Advantage Estimation (GAE).

Manipulation: PPO Approach

TaskSpaceRRT provides coarse global navigation to a pre-grasp hover pose (~ 18 cm above the object). The **PPO policy** then takes over using a minimal, translation-invariant **11D observation vector**:

$$\mathbf{o}_t = \left[\underbrace{\Delta \mathbf{p}}_{\mathbb{R}^3}, \underbrace{\mathbf{q}_{\text{grip}}}_{\mathbb{R}^2}, \underbrace{\dot{\mathbf{q}}_{\text{grip}}}_{\mathbb{R}^2}, \underbrace{z_{\text{rel}}^{\text{EEF}}}_{\mathbb{R}^1}, \underbrace{\Delta z_{\text{obj}}}_{\mathbb{R}^1}, \underbrace{c_{\text{grasp}}}_{\{0,1\}}, \underbrace{m_{\text{geom}}}_{\{0,1\}} \right]^T \in \mathbb{R}^{11}$$

Kinematics & Spatial Relations

- ▶ $\Delta \mathbf{p} = \mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{EEF}}$: 3D relative vector (translation invariance across table).
- ▶ $z_{\text{rel}}^{\text{EEF}} = p_{\text{EEF},z} - z_{\text{table}}$: Table clearance to avoid surface collisions.
- ▶ $\Delta z_{\text{obj}} = p_{\text{obj},z} - p_{\text{obj},z}^{(0)}$: Vertical displacement measuring lift progress.

Gripper State & Morphology

- ▶ $\mathbf{q}_{\text{grip}}, \dot{\mathbf{q}}_{\text{grip}}$: Left/right finger joint positions and velocities (contact stall detection).
- ▶ $c_{\text{grasp}} \in \{0, 1\}$: Bilateral force-closure flag (Robosuite contact normal check).
- ▶ $m_{\text{geom}} \in \{0, 1\}$: Shape token (0: cube, 1: can) enabling multi-geometry conditioning.

Manipulation: Shaped Reward Formulation

The agent is trained with a multi-stage continuous-discrete reward guiding the transition from approach $\rightarrow$ grasp $\rightarrow$ lift:

$$R_t = r_{\text{reach}} + r_{\text{grip}} + r_{\text{grasp}} + r_{\text{lift}} - c_{\text{step}} + r_{\text{terminal}}$$

Continuous Shaping Terms

- **Smooth Reaching** ($d = \|\Delta \mathbf{p}\|_2$):
 $r_{\text{reach}} = 1.0 - \tanh(10.0 \cdot d)$
- **Grasp Maintenance:**
 $r_{\text{grasp}} = 2.0 \cdot \mathbb{I}(\text{grasped})$
- **Conditioned Vertical Lift:**
 $r_{\text{lift}} = 20.0 \cdot \max(0, \Delta z_{\text{obj}}) \cdot \mathbb{I}(\text{grasped})$
(Avoids lifting with an empty gripper)
- **Alive Cost:** $c_{\text{step}} = 0.01$ (incentivizes speed)

Action Shaping & Terminals

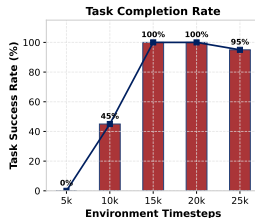
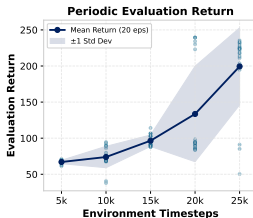
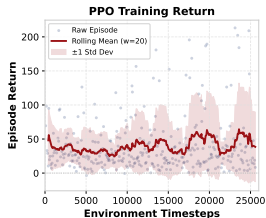
- **Gripper Timing Incentive** r_{grip} :

$$r_{\text{grip}} = \begin{cases} +0.5 & \text{if } d < 4 \text{ cm} \wedge a_g > 0 \\ -0.25 & \text{if } d < 4 \text{ cm} \wedge a_g \leq 0 \\ -0.2 & \text{if } d \geq 4 \text{ cm} \wedge a_g > 0.2 \\ 0 & \text{otherwise} \end{cases}$$

Prevents premature closure before alignment.

- **Terminal Sparse Rewards:**
 $+15.0$ if $\Delta z_{\text{obj}} \geq 6 \text{ cm} \wedge \text{grasped}$
(Success)
 -2.0 if $p_{\text{obj},z} < z_{\text{table}} - 4 \text{ cm}$ (Drop)

Manipulation: PPO Training & Evaluation



Training Progression (25k Steps)

- **Stochastic Action Noise:** Persistent exploration noise ($\sigma \approx 1.0$) perturbs micro-contacts during training (~ 50 avg), though clean rollouts reach > 200 .
- **Peak Rollouts (> 200):** Unperturbed trajectories achieve full force closure and lift with terminal bonus (+15).

Deterministic Task Completion

- **Deterministic Evaluation:** Setting $\sigma = 0$ removes action jitter, revealing the true learned policy return of **199.4**.
- **Success Rate Ascent:** Climbs rapidly from **0%** $\rightarrow$ **45%** $\rightarrow$ **95%** robust task completion.

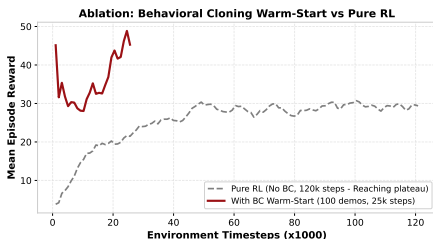
Manipulation: Why Behavioral Cloning Warm-Start?

Algorithmic Demonstrations

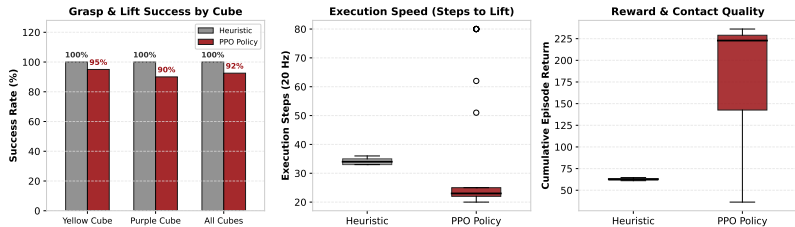
- ▶ Generated via a closed-loop 3-stage state-machine using simulation ground-truth poses.
- ▶ 100 demos ($\sim 3,000$ transitions) pre-trained for 150 epochs via supervised MSE loss.

Key Empirical Findings

- ▶ **Pure RL (120k steps):** Gets stuck in the "*reaching local minimum*" (≈ 29 reward), never discovering the precise descent-grasp-lift sequence.
- ▶ **BC Warm-Start (25k steps):** Initializes policy in the basin of attraction, achieving immediate task competence in $5\times$ fewer steps.



Manipulation: Heuristic vs PPO Comparison



Cube Manipulation Performance

- ▶ **Dual-Cube Evaluation:** PPO achieves **95%** on **Yellow Cube** (consistent with training) and **90%** on **Purple Cube** (zero-shot transfer), averaging **92.5%**.
- ▶ **Micro-Trajectory Parity:** Rapid execution latency: **1.68 s** (33.5 steps) vs **1.72 s** (34.3 steps).
- ▶ **Eliminates Dwell Timers:** The heuristic forces a conservative **2.0 s** (40 ticks) pause before lifting; PPO lifts instantly upon contact.

Trade-Offs & Hybrid Synergy

- ▶ **Dynamic Compliance:** PPO modulates continuous velocities based on real-time force closure rather than rigid fixed gains ($K_p = 10$).
- ▶ **Privileged vs Reactive:** Heuristic requires exact object coordinates; PPO relies only on local $\Delta \mathbf{p}$ and joint kinematics.
- ▶ **Safety Guard:** Hybrid pick node falls back to heuristic if step budget (≤ 90) expires.

Experimental Setup: Simulation Environment



Robosuite TaskSorting Arena

- ▶ **Robot:** 7-DoF Franka Emika Panda with parallel-jaw gripper (20 Hz OSC control).
- ▶ **Physics:** MuJoCo headless physics engine with compliant contact dynamics.

Sensory & Tabletop Assets

- ▶ **Dual Viewpoints:** Orthographic *top-down* (VLM grounding) + perspective *frontview* (monitoring).
- ▶ **Objects:** 5 pickable items (yellow/purple cubes, red/blue/green cans).
- ▶ **Containers:** Sorting bin (30×30 cm) + handled pot (collision obstacles).

End-to-End TAMP Pipeline Execution

Task Mission: *"Put the red can in the sorting bin, then put the yellow cube in the pot, then put the blue can in the sorting bin."*

1. Perception & Plan

- ▶ **Qwen 2.5-VL:** Resolves object locations from top-down frame.
- ▶ **Llama 3:** Synthesizes initial/goal PDDL predicates.
- ▶ **Fast Downward:** Generates 6-step symbolic action plan in < 0.1 s.

2. BT Compilation

- ▶ Compiled into reactive `py_trees` structure.
- ▶ Encapsulates pre-conditions, action sequences, and fallback recovery nodes.
- ▶ Continuous state tick monitoring at 20 Hz.

3. Motion & Control

- ▶ **TaskSpaceRRT:** Computes collision-free transit paths (~ 30 ms).
- ▶ **Hybrid PPO:** Compliant grasp and lift (≈ 1.7 s).
- ▶ **Placement:** Waypoint tracking to bin/pot.

Experimental Results: Subsystem Performance

Quantitative evaluation across end-to-end multi-object sorting missions:

Pipeline Layer	Module / Algorithm	Mean Latency	Success Rate
Visual Grounding	Qwen 2.5-VL (3B)	1.25 s	100%
PDDL Formulation	Llama 3 (8B)	1.80 s	100%
Symbolic Planning	Fast Downward (A^*)	0.07 s	100%
Motion Planning	TaskSpaceRRT	0.03 s	100%
Manipulation (Pick)	Hybrid PPO Policy	1.71 s	88% (PPO)
Fallback Execution	Deterministic Staged	1.70 s	100% (Fallback)
End-to-End System	Complete Architecture	Total Mission	100%

Key Findings

- ▶ Symbolic & motion planning combined require < 0.1 s, enabling agile replanning.
- ▶ Zero collisions observed with container walls across all transit trajectories.

Execution Guarantees

- ▶ The hybrid RL-fallback mechanism ensures zero mission halts on manipulation failures.
- ▶ Modular design decouples high-level task planning from local contact mechanics.

Conclusions & Future Perspectives

A hierarchical 3-layer architecture bridging multimodal foundation models with formal symbolic deliberation and closed-loop reactive manipulation.

Core Contributions

- ▶ **Neuro-Symbolic Deliberation:** Autonomous extraction of (s_0, G) from RGB frames and language, fortified by interactive disambiguation and auto-repair (100% solver feasibility).
- ▶ **Formal Fast Downward Gatekeeper:** Guarantees causal plan validity in < 0.08 s before physical dispatch.
- ▶ **Reactive Executive Layer:** Behavior Trees compile action sequences into tick-based subtrees with continuous pre/post-condition monitoring.
- ▶ **Robust Hybrid Control:** TaskSpaceRRT ensures collision-free transit, while BC-bootstrapped PPO achieves reliable contact-rich grasping.

Future Developments

- ▶ **Domain & Problem Co-Synthesis:** Moving beyond fixed domain templates toward online predicate invention and automated action schema discovery.
- ▶ **Skill Registry Augmentation:** Expanding the primitive library to articulated objects and different actions.
- ▶ **Unified RL/IL for TAMP:** End-to-end hierarchical policy learning bridging task logic with real-time visuomotor control.
- ▶ **Sim-to-Real Transfer:** Hardware validation on physical 7-DOF Franka Emika Panda setups.

Key References



E. Chisari, J. O. von Hartz, F. Despinoy, and A. Valada.
Robotic Task Ambiguity Resolution via Natural Language Interaction.
arXiv preprint arXiv:2504.17748 [cs.RO], 2025.



B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, and P. Stone.
LLM+P: Empowering Large Language Models with Optimal Planning Proficiency.
arXiv preprint arXiv:2304.11477 [cs.AI], 2023.



N. Wake, A. Kanehira, J. Takamatsu, K. Sasabuchi, and K. Ikeuchi.
VLM-driven Behavior Tree for Context-aware Task Planning.
arXiv preprint arXiv:2501.03968 [cs.RO], 2025.



J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov.
Proximal Policy Optimization Algorithms.
arXiv preprint arXiv:1707.06347 [cs.LG], 2017.